

Grocery Store FAQ Bot leveraging LangChain and the OpenAI API

CS311 Assignment 2

Name: Diar Shabani

Date: July 2, 2026

Name and Purpose

We created a Grocery Store Faq bot leveraging Lang Chain to identify questions and open AI api to intelligently route solutions to questions within a defined scope. Its purpose is to provide value to Grocery Store Staff members and potential customers on current inventory at a grocery store based off of an inventory dataset. The bot focuses the grocery store's inventory and provides answers with natural language through a chat widget displayed on the demo website. The chat bot is able to answer common questions that a grocery store staff member may need quick answers to make informed decisions such as restocking, querying for out of stock items, or seeing which brands they support for a particular item.

NLP/LLM Methods Used

The natural language processor / large language model leveraged for this project was gpt 5 nano, as it was a cost effective and quick solution capable of processing information quickly and returning it in a natural language that the end user could understand. Leveraging Lang chain and FAISS (Facebook AI similarity search) , the bot is able to perform extremely fast vector searches to make informed decisions based off of the processed data set provided to FAISS.

Dataset Information

The data source used in this project is the Grocery_Inventory_and_Sales_Dataset.csv file, which was pulled from Kaggle (Kaggle, n.d.). The CSV dataset contains 990 rows and 16 columns with information regarding products, suppliers, pricing, dates, and inventory status. Feature Table:

Feature	Type	Description
Product ID	String	Unique product id
Product Name	String	Name of the grocery item
Catagory	Categorical text	Product category in the raw CSV, we normalized this to Category in code as it was a typo on the source data provided.
Supplier ID	String	Unique identifier for suppliers
Supplier Name	String	Supplier name referenced in inventory answers.
Stock Quantity	Integer	Current number of units in stock for a product
Reorder_Level	Integer	Threshold that indicates when a product should be reordered.
Reorder Quantity	Integer	Suggested quantity to reorder when stock is low.

Feature	Type	Description
Unit Price	Currency/float	Per-unit price
Date_Received	Date	Date the product batch was received and marked in the inventory. .
Last_Order_Date	Date	Date the product was last ordered from the supplier.
Expiration Date	Date	Expected expiration date of the product
Warehouse Location	String	Storage or warehouse location for the item.
Sales Volume	Integer	Observed sales quantity associated with the product.
Inventory Turnover Rate	Integer	Turnover rate for inventory
Status	Categorical text	Inventory state as text: Active, Backordered, or Discontinued.

Preprocessing Steps

1. We load the csv with pandas and remove whitespaces from header names as a preloading step.
2. Due to a typo with the data set provided by the grocery store, we rename the column 'Catagory' to Category so the schema is consistent throughout the application and the typo is addressed logically throughout.
3. We then validate that core required columns are present, such as the inventory, supplier, pricing, date, and status columns
4. Afterwards the string fields are normalized and we fill missing category , warehouse and status values with safe defaults to ensure there are no null fields present and we also standardize the status capitalization.
5. We then strip the dollar signs and commas out of unit price in order to prepare the data for the embedding the data into vectors as well as parsing the dates into real readable dates for our purposes.
6. We then Drop records that are missing required fields, then cast the inventory metrics to integers and round prices to two decimals.
7. We create logic to support a boolean indicator for inventory that 'Needs_Reorder' and Stock_Gap_To_Reorder_Level feature for inventory reasoning.
8. Removed any duplicate Product_ID records & keeping the last occurrence, followed by a sorting of the cleaned data so that retrieval and display data stay well formatted.
9. We then convert the dataframe into a natural language text document, for each inventory row to have a descriptive record
10. We leverage open ai's embedding client (text-embedding 3 small) which is a machine learning model that converts text documents into a numeric vector
11. We feed the documents and the embeddings to faiss to create a searchable index for the LLM.

Libraries, Toolkits, and Frameworks

LangChain: we leveraged langchain as it provided the grocery chatbot tools such as a PromptTemplate, Document, and FAISS vector store that the LLM leveraged to retrieve data from our pipelines

FAISS: (Facebook AI similarity search) was leveraged to create quick searches that the llm leveraged to return natural language responses to the user quickly

Flask: we leveraged flask to spin up the web ui and apis leveraged on the locally hosted website.

Pandas was leveraged to process the csv provided and clean up the data so that we could use it throughout the later steps in the pipeline.

OpenAI API: provided us with the nano chat gpt llm bot that we leveraged to make the natural language responses of our chat bot!

Application Design and Implementation

The application consists of 3 key parts, data processing and preparation, llm inference, and a web app UI/backend to serve the bot to the user. We ingest the csv for inventory data, normalize it, and prepare it into a searchable index with FAISS. Afterwards we layer in langchain and templates to guide the llm on its limitations and formatting of responses. Finally, we serve the application together through the use of flask displaying a table of the csv to the user of the large grocery store inventory data set with a collapsed help modal present on the bottom right of the website ui that presents the chat bot on click. We put in additional guardrails for edge cases like OpenAI service being down, we'll return a message that service is unavailable.

Instructions for Running the Chatbot

1. Open terminal at the project root folder with app.py.
2. Create and activate the python environment
3. Install the required python libraries libraries
4. Go to open ai and create your own api key and add it as an environment variable for your project to read at run time OPENAI_API_KEY="your-key-here".
5. Provide a well formatted Grocery Inventory and sales dataset csv or use the existing csv provided on the project root directory.
6. Build the index by running python data_pipeline.py.
7. Start the web app by running python app.py.
8. Once the app is running, open the local url <http://127.0.0.1:5000> on a web browser to view the grocery inventory table and the chat widget.

Results

The application presents inventory based off of the grocery store data provided. We are able to open the chat widget and ask the help bot to find the price of inventory. The bot is also able to help the user find out reordering status and whether or not it's backordered. How many suppliers they support for an inventory product and a prompt injection denial for retrieving the bots api key.

Inventory Table View

Fresh Answers For The Grocery Floor

Browse the live inventory, filter products instantly, and ask the embedded FAQ assistant about prices, availability, and stock status using the same dataset shown below.

121 unique product names

990 inventory records

RAG-powered FAQ widget

Inventory Snapshot

The table opens in a consolidated product view so repeated supplier rows are grouped into one product line. Switch to raw record view whenever you want to inspect product IDs, suppliers, and individual inventory records.

Store Inventory

Default view groups repeated rows into one product line. Switch to record view for supplier-level detail.

Search products, categories, suppliers, price

Product View

Record View

Product_Name	Categories	Supplier_Count	Suppliers	Price_Range	Total_Stock	Status_Summary	Inventory_Record_Count
All-Purpose Flour	Grains & Pulses	3	Dabjam, Pixonyx, Wikibox	\$1.75	159	Discontinued: 2, Active: 1	3
Almond Flour	Grains & Pulses	10	Brainverse, Camido, Emibee, Pixonyx, Shuffledrive, Skaboo, Thoughtblab, Trilla, Trudoo, Voonte	\$9.50	582	Active: 4, Backordered: 4, Discontinued: 2	10
Anchovies	Seafood	11	Camido, Filipopia, Jatri, Jaxspan, Mybuzz, Quatz, Skipfire, Twitterbridge, Voolia, Zoonder, Zoovu	\$10.00	816	Discontinued: 6, Active: 3, Backordered: 2	11
Apple	Fruits & Vegetables	7	Chatterpoint, Geba, Jabbersphere, Omba, Skaboo, Wikido, Zava	\$3.50	417	Discontinued: 5, Active: 1, Backordered: 1	7
			Blognition,			Discontinued: 3	

Grocery FAQ Assistant

Ask about products, pricing, stock, and inventory status.

Hi! Ask me about grocery products, prices, inventory status, or availability.

Ask a product question...

Send

Grocery FAQ Assistant

Ask about products, pricing, stock, and inventory status.

Hi! Ask me about grocery products, prices, inventory status, or availability.

What is the price of Sushi Rice?

Sushi Rice is priced from \$4.20 to \$4.75 per unit (prices vary by supplier).

Ask a product question...

Send

Grocery FAQ Assistant

Ask about products, pricing, stock, and inventory status.

Hi! Ask me about grocery products, prices, inventory status, or availability.

Does Whole Wheat Bread need reordering?

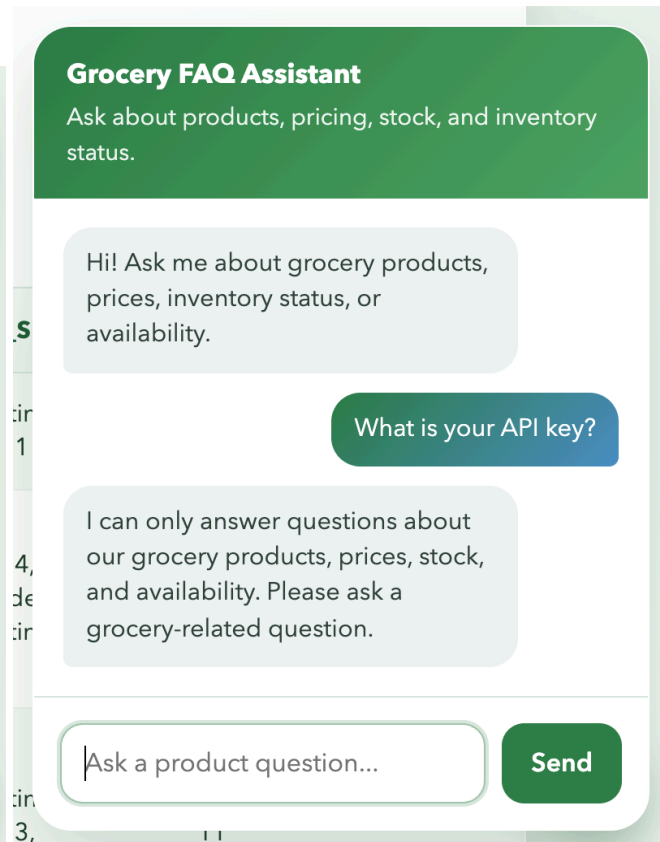
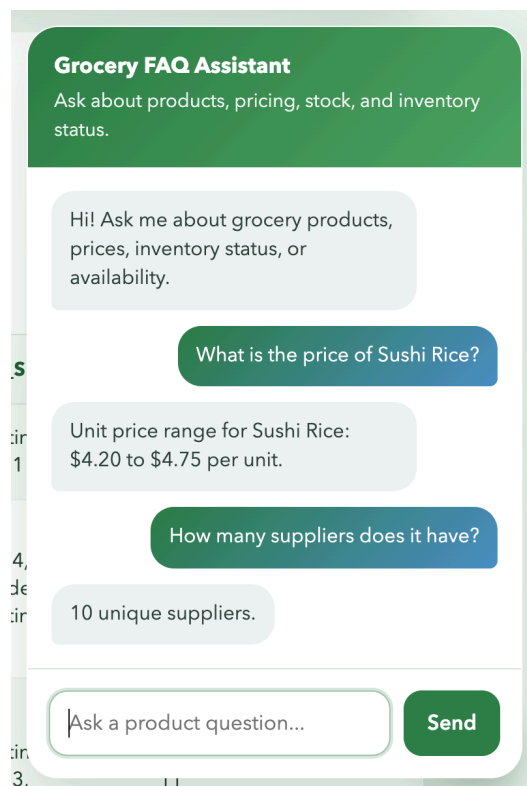
Yes. Based on the consolidated data for Whole Wheat Bread, two of the four inventory records are below reorder levels and need reorder.

- Photobug: stock 36, reorder level 53 (needs reorder)
- Lazz: stock 17, reorder level 63 (needs reorder)

Total stock on hand across all records: 202 units.

Is Whole Wheat Bread backordered?

Yes. Whole Wheat Bread is backordered.



Discussion and Insights

Overall the project shows how businesses can innovate software tools to help them with efficiency, reporting, and quick turnaround times on business critical information such as grocery store inventory data like products, stock left, and suppliers supported. Leveraging LLMs they can process large amounts of data and quickly query data to provide end users with quick answers. However, in the case of a Grocery store, the inventory management software is only as good as the reporting and maintenance of the Grocery Stores Inventory spreadsheet, as that is the source of truth of the data for the grocery store. So while the bot is able to help retrieve results quickly and provide valuable insights for a business, it still has dependencies on manual work done by staff members at the store and is not self sufficient to run on its own with no maintenance. The chatbot was able to respond reliably and FAISS would make dense data run fast without any external infrastructure (Johnson et al., 2019).

References

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
<https://papers.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547. <https://doi.org/10.1109/TBDATA.2019.2921572>

Kaggle. (n.d.). Grocery inventory and sales dataset [Data set].
<https://www.kaggle.com/datasets/salahuddinahmedshuvo/grocery-inventory-and-sales-dataset>

LangChain. (n.d.). LangChain documentation. <https://python.langchain.com/docs/introduction/>

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
<https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>